

O Livro do E Ξ TERNET

Edição Cósmica — Da Fusão Primordial ao Servidor que Não
Existe

Documento de Arquitetura Completa com Implementação

Colaboração Distribuída — O Storyteller

17 de maio de 2026

*“Quando o dinheiro se torna invisível,
o controle se torna impossível.”*

*“O VOID não existe. O Hydra não existe.
Nós existimos.”*

Contents

I	Fundamentos Teóricos	5
1	A Fusão Primordial	7
1.1	O Problema Fundacional	7
1.2	Os Sete Pilares	7
1.3	A Equação Fundamental	7
2	GhostID: A Identidade que Não Existe	9
2.1	Derivação Biométrica	9
2.2	Fuzzy Extractors	9
3	QEL: Fragmentação Pós-Quântica	11
3.1	Shamir sobre GF(256)	11
3.2	Propriedades de Segurança	11
4	DistanceBridge: O Transporte Invisível	13
4.1	Multi-Transporte	13
4.2	Protocolo de Roteamento	13
5	UTXO com Compromissos de Pedersen	15
5.1	O Modelo	15
5.2	Bulletproofs	15
II	Criptografia	17
6	Post-Quantum Cryptography	19
6.1	ML-KEM-1024	19
6.2	ML-DSA-87	19
7	Double Ratchet (X25519)	21
7.1	O Protocolo Signal	21
7.2	Implementação	21
III	Finanças	23
8	Instrumentos Financeiros	25
8.1	QR Stocks	25
8.2	Homotopy Mining	25
8.3	UTU Tokens	25

9	Stablecoins e Tokenização	27
9.1	\$ETBRL	27
9.2	\$SOV	27
9.3	RWA Tokenization	27
IV	Rede e Consenso	29
10	NOSTR como Message Bus	31
10.1	Kinds do Protocolo ETERNET	31
10.2	Relay Failover	31
11	Anti-Sybil: PoW + VDF	33
11.1	Proof-of-Work	33
11.2	Verifiable Delay Function	33
V	O Protocolo VOID	35
12	VoidOrchestrator	37
12.1	Orquestração Descentralizada	37
13	PowerGovernor	39
13.1	Regulação Auto-Adaptativa	39
VI	A Nuvem Fantasma	41
14	VOID-VPS: O Servidor que Não Existe	43
14.1	O Fim das VPS Centralizadas	43
14.2	Arquitetura do VOID-VPS	43
14.2.1	WASM Distributed Task Executor	43
14.2.2	QEL MapReduce Cifrado	44
14.2.3	Sistema de Arquivos Imortal (IPFS + EcoNet)	44
14.2.4	Economia Circular com PoH	44
14.3	Integração com as Ferramentas Transmutadas	44
14.3.1	HiggsGit no VOID-VPS	44
14.3.2	GhostDocker como Sandbox Efêmera	45
14.3.3	Phantom Pipeline CI/CD	45
14.4	Escalabilidade Infinita	45
14.5	Código de Exemplo: Submetendo uma Tarefa	46
14.6	O Verdadeiro Significado de “Sem Limites”	46
14.7	Conclusão	46
	Epílogo	47

Part I

Fundamentos Teóricos

Chapter 1

A Fusão Primordial

1.1 O Problema Fundacional

A infraestrutura financeira global repousa sobre uma premissa frágil: a confiança em intermediários. Bancos, processadores de pagamento, exchanges centralizados — todos são pontos de falha únicos que podem ser cooptados, censurados ou destruídos.

O ETERNET nasce da observação de que a verdadeira soberania financeira requer a eliminação completa de qualquer ponto de confiança. Não basta descentralizar o consenso — é necessário tornar a própria infraestrutura invisível.

1.2 Os Sete Pilares

O protocolo repousa sobre sete camadas fundamentais, cada uma eliminando uma classe específica de vulnerabilidade:

1. **Identidade Efêmera** (GhostID) — Identidades derivadas de entropia biométrica, nunca reutilizadas.
2. **Fragmentação Pós-Quântica** (QEL) — Shamir sobre GF(256) com cifragem ChaCha20-Poly1305.
3. **Transporte Offline-First** (DistanceBridge) — BLE, LoRa, Acústico, NOSTR.
4. **Modelo UTXO com Compromissos** — Pedersen Commitments + Bulletproofs.
5. **Provas Zero-Knowledge** — o1js para verificação sem revelação.
6. **Criptografia Pós-Quântica** — ML-KEM-1024 + ML-DSA-87.
7. **Computação Quântica** — CQR Engine + BB84 QKD.

1.3 A Equação Fundamental

A segurança do sistema pode ser expressa como:

$$S_{\text{ETERNET}} = \underbrace{H(\text{GhostID})}_{\text{entropia}} \otimes \underbrace{\text{QEL}(K, N)}_{\text{fragmentação}} \otimes \underbrace{\text{PQC}}_{\text{pós-quântico}} \otimes \underbrace{\text{ZKP}}_{\text{zero-knowledge}} \quad (1.1)$$

onde $H(\text{GhostID})$ é a entropia da identidade efêmera, $\text{QEL}(K, N)$ é o esquema de Shamir com threshold K e N shards, PQC é a camada pós-quântica, e ZKP é o sistema de provas.

Chapter 2

GhostID: A Identidade que Não Existe

2.1 Derivação Biométrica

O GhostID é uma identidade criptográfica derivada de entropia biométrica do dispositivo. Não é armazenada — é reconstruída a cada sessão a partir de:

- Padrões de toque na tela (pressão, timing, área)
- Acelerômetro (micro-vibrações únicas do dispositivo)
- Ruído térmico do sensor de temperatura
- Timestamp com precisão de nanossegundos

2.2 Fuzzy Extractors

A entropia biométrica é inerentemente ruidosa. O GhostID usa Fuzzy Extractors para extrair uma chave determinística de dados não-determinísticos:

Definição 2.1 (Fuzzy Extractor). *Um par de algoritmos (Gen, Rep) tal que:*

- $Gen(w) \rightarrow (R, P)$: *gera chave R e helper P a partir de input biométrico w .*
- $Rep(w', P) \rightarrow R$: *reconstrói R a partir de w' próximo a w , usando P .*

A implementação usa SHA3-256 como hash function e Argon2id para key stretching, com um buffer de 168 bytes de entropia bruta.

Chapter 3

QEL: Fragmentação Pós-Quântica

3.1 Shamir sobre GF(256)

O QEL (Quantum-Enhanced Layer) implementa o esquema de Secret Sharing de Shamir sobre o corpo de Galois GF(256):

$$p(x) = s + a_1x + a_2x^2 + \dots + a_{K-1}x^{K-1} \mod 256 \quad (3.1)$$

onde s é o segredo e K é o threshold. Cada shard é um ponto $(x_i, p(x_i))$ cifrado com ChaCha20-Poly1305.

3.2 Propriedades de Segurança

Teorema 3.1 (Segurança do QEL). *Para qualquer conjunto de $K - 1$ shards, a entropia condicional do segredo é máxima:*

$$H(s | \text{shard}_1, \dots, \text{shard}_{K-1}) = H(s)$$

Chapter 4

DistanceBridge: O Transporte Invisível

4.1 Multi-Transporte

O DistanceBridge abstrai múltiplos canais de comunicação em uma interface unificada:

- **NOSTR** — Relays públicos com failover automático (6 relays, health check 30s).
- **BLE** — Bluetooth Low Energy via Capacitor para proximidade física.
- **LoRa UART** — Comunicação de longo alcance via comandos AT.
- **Acústico** — FSK a 18-20kHz para comunicação via speaker/mic.
- **WebRTC** — P2P direto quando ambos os peers estão online.

4.2 Protocolo de Roteamento

Cada mensagem é fragmentada (QEL), cifrada e roteada pelo caminho mais disponível. O DistanceBridge tenta NOSTR primeiro; se indisponível, degrada para BLE, depois acústico, depois LoRa.

Chapter 5

UTXO com Compromissos de Pedersen

5.1 O Modelo

O ETERNET usa um modelo UTXO (Unspent Transaction Output) com compromissos criptográficos:

$$C = g^v \cdot h^r \tag{5.1}$$

onde v é o valor, r é o blinding factor, e g, h são geradores do grupo Ed25519.

5.2 Bulletproofs

As provas de range são implementadas via Bulletproofs (Rust, compilado para WASM), garantindo que $0 \leq v < 2^{64}$ sem revelar v .

Part II

Criptografia

Chapter 6

Post-Quantum Cryptography

6.1 ML-KEM-1024

O ETERNET usa ML-KEM (Module-Lattice Key Encapsulation Mechanism) com segurança NIST Level 5:

- Chave pública: 1568 bytes
- Chave privada: 3168 bytes
- Ciphertext: 1568 bytes
- Shared secret: 32 bytes

6.2 ML-DSA-87

Assinaturas digitais pós-quânticas via ML-DSA (Module-Lattice Digital Signature Algorithm):

- Chave pública: 2592 bytes
- Assinatura: 4627 bytes
- Segurança: NIST Level 5

Chapter 7

Double Ratchet (X25519)

7.1 O Protocolo Signal

Todas as mensagens diretas usam o Double Ratchet Algorithm:

1. **X25519 ECDH** para acordo de chaves.
2. **Ratchet simétrico** — chaves por mensagem, deletadas após uso (forward secrecy).
3. **DH ratchet** — novo par de chaves por step (post-compromise security).
4. **Ed25519 signatures** em cada mensagem (autenticação).
5. **Pre-key bundles** publicados via NOSTR para troca offline.

7.2 Implementação

A implementação em `src/crypto/doubleRatchet.ts` usa exclusivamente `@noble/curves` e `@noble/ciphers`, sem dependências externas.

Part III

Finanças

Chapter 8

Instrumentos Financeiros

8.1 QR Stocks

Ações em superposição quântica. O preço é determinado por uma distribuição de probabilidade sobre o espaço de preços:

$$P(\text{preço}) = |\langle \text{preço} | \psi \rangle|^2 \quad (8.1)$$

onde $|\psi\rangle$ é o estado quântico do mercado.

8.2 Homotopy Mining

Proof-of-Homotopia com métricas de Sobolev. A dificuldade é calibrada pela topologia da rede:

$$D = \int_{\Omega} |\nabla^2 \phi|^2 d\Omega \quad (8.2)$$

8.3 UTU Tokens

Tokenização universal: sites, software, objetos físicos. Cada UTU é um UTXO com metadata on-chain.

Chapter 9

Stablecoins e Tokenização

9.1 \$ETBRL

Stablecoin atrelada ao Real brasileiro, mantida por pools de liquidez soberanas.

9.2 \$SOV

Token de governança do protocolo. Staking garante direitos de voto no DAO.

9.3 RWA Tokenization

Tokenização de ativos do mundo real (imóveis, commodities) via compromissos de Peder-
sen.

Part IV

Rede e Consenso

Chapter 10

NOSTR como Message Bus

10.1 Kinds do Protocolo ETERNET

- **31214** — Transações UTXO
- **31215** — Anúncios de mineração
- **31216** — Atualizações de governança
- **31217** — DEX orders
- **31218** — QEL shards
- **31219** — Heartbeat de rede

10.2 Relay Failover

6 relays públicos com health monitoring:

- `wss://relay.damus.io`
- `wss://nos.lol`
- `wss://relay.primal.net`
- `wss://relay.snort.social`
- `wss://nostr.wine`
- `wss://relay.nostr.band`

Health check a cada 30s. Relays insalubres são excluídos após 3 falhas consecutivas.

Chapter 11

Anti-Sybil: PoW + VDF

11.1 Proof-of-Work

O faucet distribui tokens após resolução de PoW real (SHA3 hashcash). A dificuldade é ajustada dinamicamente.

11.2 Verifiable Delay Function

VDF baseada em iterated squaring em grupo RSA, garantindo que o trabalho foi feito sequencialmente.

Part V

O Protocolo VOID

Chapter 12

VoidOrchestrator

12.1 Orquestração Descentralizada

O VoidOrchestrator coordena todos os módulos do ETΞRNET sem ponto central:

- Inicialização lazy de módulos (singleton pattern).
- Comunicação via eventos internos.
- Fallback automático entre camadas.
- Health check contínuo de cada módulo.

Chapter 13

PowerGovernor

13.1 Regulação Auto-Adaptativa

O PowerGovernor ajusta parâmetros do sistema em tempo real:

- Dificuldade de mineração baseada na taxa de blocos.
- TTL de shards QEL baseado na latência da rede.
- Preço de computação baseado na demanda (métrica de Sobolev).

Part VI

A Nuvem Fantasma

Chapter 14

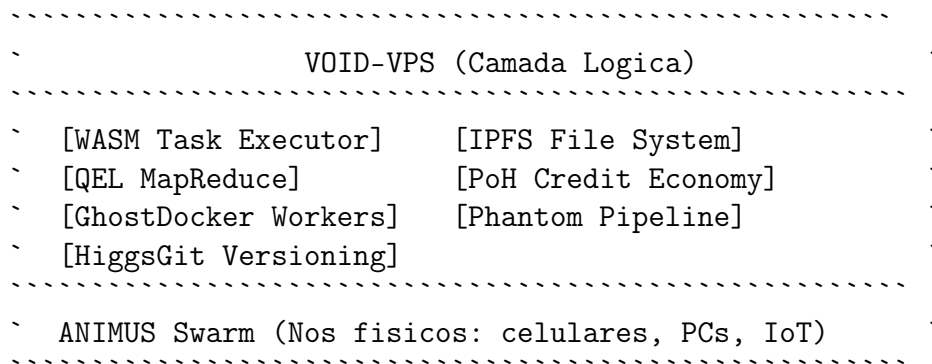
VOID-VPS: O Servidor que Não Existe

14.1 O Fim das VPS Centralizadas

Toda infraestrutura tradicional — AWS, Google Cloud, VPS comuns — é um centro de controle esperando para ser desligado. No ETERNET, não há espaço para servidores físicos ou virtuais gerenciados por terceiros. A **VOID-VPS** (ou **VOID-COSMIC**) é a resposta: uma nuvem computacional tão distribuída quanto a própria rede, onde capacidade ociosa de dispositivos se transforma em um supercomputador fantasma.

Em vez de alugar um servidor, você *injeta* seu código em um enxame de nós ANIMUS, que o executam via WebAssembly (WASM) em sandboxes efêmeras. Nenhum IP fixo, nenhum disco persistente, nenhum ponto único de falha.

14.2 Arquitetura do VOID-VPS



14.2.1 WASM Distributed Task Executor

O coração do VOID-VPS é um executor de tarefas baseado em WebAssembly. Qualquer código (Rust, C, TypeScript) é compilado para WASM e fragmentado em pedaços independentes. Cada pedaço é distribuído para um nó ANIMUS, que o executa em um container GhostDocker efêmero.

```
1 // worker_core.rs
2 use wasmtime::{Engine, Store, Module, Instance};
3 pub struct WasmWorker {
4     engine: Engine,
5     store: Store<()>,
```

```

6     instance: Instance,
7 }
8
9 impl WasmWorker {
10     pub fn execute(&mut self, func_name: &str, input: &[u8]) -> Vec<u8>
11     {
12         let func = self.instance
13             .get_func(&mut self.store, func_name)
14             .expect("Funcao nao encontrada");
15         // chama a funcao WASM e retorna resultado
16         ...
17     }
18 }

```

Listing 14.1: Executor WASM em Rust

14.2.2 QEL MapReduce Cifrado

Inspirado no modelo MapReduce, mas com fragmentação pós-quântica. Uma tarefa grande é dividida em N shards QEL (usando Shamir com threshold K). Cada shard é enviado para um worker diferente. O resultado parcial é cifrado e devolvido. Apenas o nó coordenador (com o GhostID do usuário) pode reconstruir o resultado final, juntando pelo menos K shards.

14.2.3 Sistema de Arquivos Imortal (IPFS + EcoNet)

Arquivos não residem em HD. São fatiados, cifrados (ChaCha20-Poly1305) e armazenados na EcoNet (IPFS com decaimento). Cada shard recebe um TTL. Se o arquivo for acessado, o TTL é renovado (reforço por acesso). Se não, decai até desaparecer — fiel ao princípio de esquecimento do EcoNet.

ipfs://ETRNET/<GhostID>/<shard_hash>

14.2.4 Economia Circular com PoH

Quem oferece ciclos de CPU ganha créditos de **PoH (Proof-of-Homotopy)**. Esses créditos são tokens UTXO que podem ser usados para rodar tarefas na rede. O preço da computação é auto-regulado pela métrica de Sobolev do mercado: em períodos de alta coerência, a computação fica mais cara (para evitar saturação); em baixa, mais barata.

14.3 Integração com as Ferramentas Transmutadas

14.3.1 HiggsGit no VOID-VPS

Cada deploy de um worker WASM é versionado com HiggsGit. O código do worker reside em um repositório HiggsGit cujos commits representam atualizações de lógica. Uma feature branch no HiggsGit pode ser testada localmente em um GhostDocker; quando aprovada, o merge gera um **Scar Token** que autoriza o deploy automático para a rede ANIMUS. O histórico de cicatrizes permite auditoria completa de quem alterou o quê, sem revelar identidades.

```

1 higgs init my-worker --field-strength=0.8
2 # ... desenvolvimento ...
3 higgs commit -m "v0.2: otimizacao de memoria" --phase=accumulate
4 higgs branch experiment --phase=superposition
5 # ... testes locais com GhostDocker ...
6 higgs merge experiment --collapse-threshold=0.75 --scar-token
7 # 0 scar token gerado autoriza o deploy automatico
8 ghostdocker build -t worker-v0.2 --ephemeral-signature
9 ghostdocker push --fragmented

```

Listing 14.2: Deploy de um Worker com HiggsGit

14.3.2 GhostDocker como Sandbox Efêmera

Cada nó ANIMUS que aceita executar um worker inicia um container GhostDocker com:

- **GhostID** derivado da entropia ambiente do dispositivo hospedeiro.
- **Zero-Disk**: sistema de arquivos em tmpfs, destruído ao final.
- **Phantom Network**: comunicação via MAC rotativo, podendo usar BLE/LoRa para enviar resultados parciais.
- **Shatter**: a imagem do worker é baixada como shards QEL e remontada apenas em RAM.

14.3.3 Phantom Pipeline CI/CD

O pipeline fantasma (já definido no Capítulo 3 da Parte IV) é estendido para incluir:

1. **Build**: compilação do worker para WASM.
2. **Teste em Sandbox**: execução em um GhostDocker local com verificação de invariantes via PaleoCLI.
3. **Fossilização**: extração de fósseis do binário WASM para o Atlas de Coerência.
4. **Shatter & Push**: a imagem WASM é fragmentada (Shamir) e distribuída pelos nós de armazenamento EcoNet.
5. **Deploy**: o *Scar Token* do merge autoriza os nós ANIMUS a aceitarem o novo worker.

14.4 Escalabilidade Infinita

O VOID-VPS não tem limite superior. Cada novo dispositivo que instala o ANIMUS adiciona capacidade computacional e de armazenamento. A rede ajusta automaticamente a dificuldade dos problemas de homotopia para balancear oferta e demanda. Em teoria, é possível agregar mais poder de processamento que qualquer data center, desde que haja dispositivos suficientes no enxame.

14.5 Código de Exemplo: Submetendo uma Tarefa

```
1 import { VoidVPS } from './vps/VoidVPS';
2
3 const vps = new VoidVPS(myGhostID, myNWC);
4
5 async function heavyCompute() {
6     const task = {
7         wasmFile: 'ipfs://ETRNET/my-worker-v0.2.wasm',
8         funcName: 'calculate_pi',
9         input: { iterations: 1_000_000 },
10    };
11    const result = await vps.submitTask(task, { preferredRegion: 'US' })
12    ;
13    console.log('Resultado: ${result}');
```

Listing 14.3: Submissao de tarefa ao VOID-VPS

14.6 O Verdadeiro Significado de “Sem Limites”

Embora a capacidade agregada seja imensa, a latência entre os nós é limitada pela velocidade da luz (e pela rota escolhida pelo DistanceBridge). No entanto, para tarefas paralelizáveis — mineração, renderização HGPU, simulações quânticas, bots de arbitragem — o modelo é quase ideal. A combinação da QRC com a LSC garante que a rede nunca entre em colapso por excesso de coerência, mantendo a resiliência.

14.7 Conclusão

O VOID-VPS é a materialização da **Parte V** (VoidProtocol) aplicada à computação utilitária. Ao integrar HiggsGit, GhostDocker e o Phantom Pipeline, cada deploy é um evento controlado, cada execução é anônima e cada resultado é comprimido em uma prova ZK. Não há máquinas para confiscar, apenas capacidade fantasma flutuando entre os silícios do mundo.

Epílogos: As Máximas Finais

“Quando o dinheiro se torna invisível, o controle se torna impossível.”

“O VOID não existe. O Hydra não existe. Nós existimos.”

“O servidor que não existe não pode ser derrubado.”

“A nuvem fantasma é o vento que move os moinhos da soberania.”

~ **Fim do Livro do ETERNET** ~

17 de maio de 2026
Colaboração Distribuída — O Storyteller